SVNIT SURAT

CampusConnect

A Student Campus Events & Community Portal

DATABASE MANAGEMENT SYSTEMS (CS203)

B.Tech. II CSE | Semester III | Academic Year 2026-27

DETAIL	INFORMATION
Institute	Sardar Vallabhbhai National Institute of Technology, Surat
Department	Computer Science and Engineering
Team name	[Enter registered team name]
Team members	[Name and roll number of member 1] [Name and roll number of member 2] [Name and roll number of member 3, if applicable]
Faculty / guide	[Enter if required by the department]

PROJECT REPORT

Prepared around the six phases specified in the CS203 DBMS project instructions.

PHASE GUIDE

Report structure

The report follows the project lifecycle given in the course instructions.

SECTION	WHAT IT COVERS
A. Project Requirements	Problem definition, objectives, scope, users, requirements, and high-level tasks.
B. Conceptual Design	Entities, ER model, identifiers, relationships, cardinalities, constraints, and design rationale.
C. Logical Design	Relational schema, primary and foreign keys, data dictionary, and normalization.
D. Physical Design	SQL implementation, constraints, indexes, views, CRUD statements, and design justification.
E. Prototype, Development, and Testing	Application architecture, page flows, role permissions, validation, test plan, and known limits.
F. Project Presentation	Demonstration plan, slide outline, and expected live demo sequence.
Appendices	Setup instructions, review checklist, and submission notes.

Important project note: the assignment brief requires a database application, a report covering the listed phases, web interfaces connected to SQL tables, and a presentation. It does not prescribe a specific web framework or database engine. This design uses Flask and SQLite for a lightweight local demo, and includes an equivalent MySQL 8 schema for SQL implementation in a MySQL tool if required by the course.

PHASE A

Project Requirements

Understanding the campus problem before designing the database.

A.1 Problem statement

Campus information is often distributed across messaging groups, posters, club pages, and informal student networks. A student may miss an event announcement, struggle to discover clubs, or have no single place to see registrations and notices. CampusConnect proposes one portal that organises campus events, student communities, event registrations, and announcements in a structured database-backed application.

A.2 Project objectives

- Provide students with a single interface to browse upcoming and past campus events.
- Allow authenticated students to register for events, cancel registrations, and view their registration history.
- Maintain campus clubs and student membership records without duplicating club details for each student.
- Publish announcements with audience and optional expiry information.
- Provide administrator-only controls for creating, updating, and deleting events, clubs, and announcements.
- Demonstrate relational design concepts: keys, cardinalities, integrity constraints, normalization, indexes, views, and CRUD operations.

A.3 Scope

IN SCOPE	OUT OF SCOPE FOR THIS PROTOTYPE
Student accounts and profile details	College SSO or integration with institute identity systems
Event listings, capacity, and registrations	Paid ticketing, payment gateway, or QR-based attendance
Club directory and membership status	Complex club election, finance, or approval workflows
Audience-based announcements and expiry	Push notifications, email/SMS gateway, or mobile app
Role-based administrator dashboard	Production deployment, analytics dashboards, and multi-campus tenancy

A.4 Stakeholders and user roles

ROLE	NEEDS / PERMISSIONS
Student / normal user	Register and log in; browse events, clubs, and visible notices; join/leave clubs; register/cancel events; edit their own profile.
Campus administrator	All student-facing read access plus event creation/edit/deletion, club creation, and announcement publishing/deletion.

ROLE	NEEDS / PERMISSIONS
Project team / evaluator	Inspect ER model and relational schema, run SQL scripts, verify integrity rules, and demonstrate workflows.

A.5 Functional requirements

ID	REQUIREMENT	DESCRIPTION
FR-01	Account creation	A visitor can create a student account with name, email, password, department, and study year.
FR-02	Authentication	A registered user can log in and log out; passwords are stored as hashes, not plain text.
FR-03	Event discovery	Users can browse upcoming and past events and search by title, description, or location.
FR-04	Event registration	A logged-in student can register once per event, subject to the event capacity.
FR-05	Registration cancellation	A student can cancel their active event registration; the row is retained as history.
FR-06	Club memberships	A student can join or leave a club; membership status is stored in the associative relation.
FR-07	Announcements	Users can read announcements visible to their audience and not yet expired.
FR-08	Profile management	A logged-in student can update their name, department, and year.
FR-09	Admin event management	An administrator can create, edit, and delete events.
FR-10	Admin content management	An administrator can create clubs and publish/delete announcements.

A.6 Non-functional requirements

- Usability: consistent navigation, readable layouts, responsive design for laptop and mobile-sized screens.
- Integrity: primary keys, unique constraints, foreign keys, and CHECK constraints protect the stored data.
- Security baseline: password hashing, parameterised SQL queries, role checks on protected routes, and HTTP-only session cookies.
- Maintainability and portability: templates, CSS, JavaScript, routes, and SQL are separated; SQLite runs locally and MySQL DDL is supplied.
- Performance: indexes support event-date filters, club/member lookups, and per-user registration history.

A.7 High-level tasks

- Confirm scope, actors, user requirements, and entities.
- Prepare the ER model and normalized schema with justified constraints.
- Implement SQL and the student/administrator workflows.
- Test, record evidence, finalize the report, and rehearse the presentation.

PHASE B

Conceptual Design

Entities, relationships, cardinalities, and rules captured before table implementation.

B.1 ER model

The conceptual model contains six core entities/relations: Users, Clubs, Events, Announcements, ClubMemberships, and EventRegistrations. ClubMemberships resolves the many-to-many relationship between users and clubs. EventRegistrations resolves the many-to-many relationship between users and events and stores registration-specific attributes.

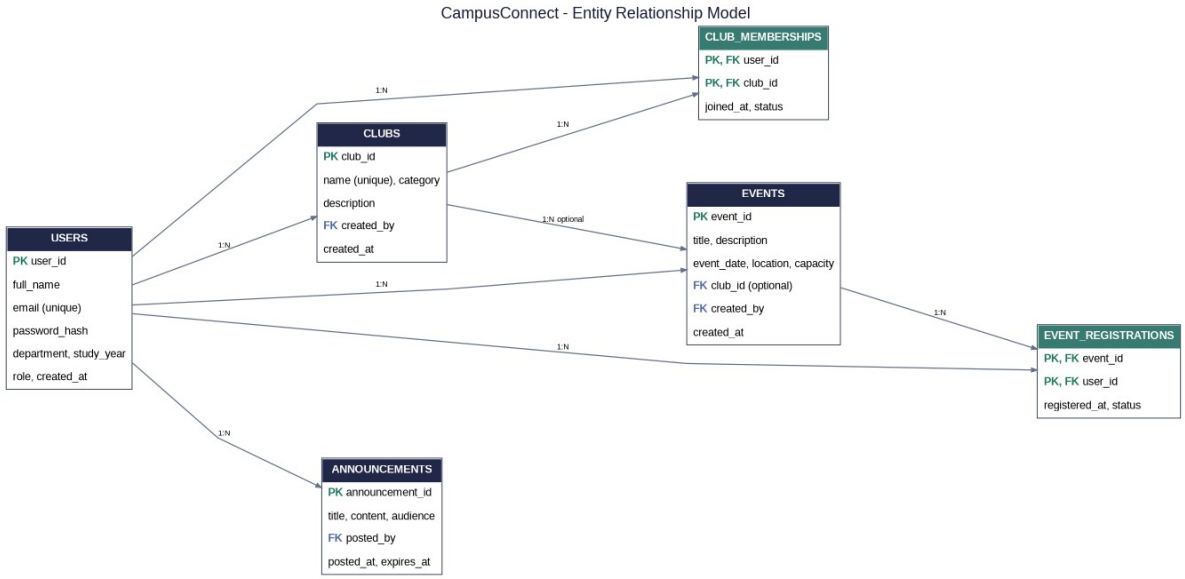


Figure B.1. CampusConnect entity-relationship model (keys and foreign-key links are labelled).

B.2 Entities and identifying attributes

ENTITY	IDENTIFIER	KEY DESCRIPTIVE ATTRIBUTES	WHY IT IS SEPARATE
Users	user_id	full_name, email, password_hash, department, study_year, role	One account record per person; role allows restricted administrator functions.
Clubs	club_id	name, category, description	Stores club details once even when many students join.
Events	event_id	title, description, event_date, location, capacity	Each event is independently managed and can be hosted by an optional club.
Announcements	announcement_id	title, content, audience, posted_at, expires_at	Separates notices from events because they have different lifecycles and audiences.

ENTITY	IDENTIFIER	KEY DESCRIPTIVE ATTRIBUTES	WHY IT IS SEPARATE
ClubMemberships	(user_id, club_id)	joined_at, status	Associative entity for the many-to-many user-club relationship.
EventRegistrations	(event_id, user_id)	registered_at, status	Associative entity for the many-to-many user-event relationship.

B.3 Relationship cardinalities

RELATIONSHIP	CARDINALITY	OPTIONALITY / INTERPRETATION
User creates Club	1 : N	One administrator/user can create many clubs; each club has zero or one recorded creator.
User organises Event	1 : N	One administrator/user can create many events; an event may retain a null creator if the creator is removed.
Club hosts Event	1 : N	A club may host many events; an event may be campus-wide and have no host club.
User joins Club	M : N via ClubMemberships	A user can join many clubs; each club can include many users. Composite PK prevents duplicate membership rows.
User registers for Event	M : N via EventRegistrations	A user can register for many events; an event can have many attendees. Composite PK prevents duplicate active/history rows per pair.
User publishes Announcement	1 : N	One user can publish many notices; a notice has zero or one stored author.

B.4 Integrity constraints and business rules

- Entity integrity: every base table has a primary key; the two associative tables use composite primary keys.
- Uniqueness: users.email is unique (case-insensitive in SQLite); clubs.name is unique.
- Referential integrity: memberships reference existing users and clubs; registrations reference existing users and events; creators/authors reference users.
- Domain constraints: role is student/admin; membership status is active/left; registration status is registered/cancelled; audience is all/students/admins.
- Capacity constraint: event capacity must be between 1 and 10,000; registration is permitted only when active registrations are below capacity.
- Temporal rules: a student cannot register for an event that has already started; expired announcements are excluded from the notice board.
- Authorisation rule: only an administrator may create/update/delete events, create clubs, or publish/delete announcements.
- Delete behaviour: registrations and memberships cascade when their parent user/event/club is deleted; creator references become NULL where the history can remain.

B.5 Design rationale

Users, clubs, events, and announcements have distinct attributes and lifecycles, so each is represented separately. The many-to-many relationships require associative tables because storing a list of club IDs or event IDs in a single column would violate relational atomicity. Registration and membership status belongs on the relationship itself rather than on either endpoint. Event capacity and registration count are not stored redundantly; the count is derived from event_registrations to prevent inconsistencies.

B.6 Modelling assumptions

ASSUMPTION	DESIGN CONSEQUENCE
One account per email address	Email is a unique candidate key and is used for login.
Each event represents one scheduled start time	A new event row is required for a separate session; multi-session events are outside the current scope.
Students may join several clubs	A user-club many-to-many relation is required.
A student has one registration state per event	The event/user pair is the composite key; cancellation changes status rather than adding duplicate rows.
Each announcement has one audience category	The prototype supports all, students, or admins rather than arbitrary custom audiences.
Only administrator accounts manage campus content	Role authorization is checked in protected backend routes.

PHASE C

Logical Design

Relational mapping, keys, and normalization.

C.1 Relational schema

```
USERS(user_id PK, full_name, email UNIQUE, password_hash, department, study_year, role, created_at)
CLUBS(club_id PK, name UNIQUE, category, description, created_by FK -> USERS.user_id, created_at)
CLUB_MEMBERSHIPS(user_id PK/FK -> USERS.user_id, club_id PK/FK -> CLUBS.club_id, joined_at, status)
EVENTS(event_id PK, title, description, event_date, location, capacity,
    club_id FK -> CLUBS.club_id, created_by FK -> USERS.user_id, created_at)
EVENT_REGISTRATIONS(event_id PK/FK -> EVENTS.event_id, user_id PK/FK -> USERS.user_id, registered_at, status)
ANNOUNCEMENTS(announcement_id PK, title, content, audience, posted_by FK -> USERS.user_id, posted_at, expires_at)
```

C.2 Relational design decisions

DESIGN DECISION	REASON
Surrogate integer identifiers for main entities	Stable compact primary keys avoid using editable descriptive attributes as identity.
Unique email and club name	Protects against accidental duplicate account identifiers and duplicate club names.
Composite keys on associative tables	A user-club pair or user-event pair is represented at most once, while status records whether the relationship is active.
Nullable club_id on Events	Supports events hosted by the campus centrally rather than by a particular club.
Nullable creator/author foreign keys	Allows content to remain if the creating account is removed; personal data handling may require separate retention policy.
Status instead of destructive cancellation	Preserves registration history and enables reactivation of a prior registration.
Derived registration counts	Avoids denormalised counters that can drift from the registration relation.

C.3 Data dictionary

Users

ATTRIBUTE	TYPE	KEY / CONSTRAINT	NULL?	MEANING
user_id	INTEGER / BIGINT	PK, auto-generated	No	Internal user identifier.
full_name	VARCHAR/TEXT	CHECK length 2-100	No	Name displayed in the portal.
email	VARCHAR/TEXT	UNIQUE	No	Login identifier.

ATTRIBUTE	TYPE	KEY / CONSTRAINT	NULL?	MEANING
password_hash	VARCHAR/TEXT	-	No	Password hash generated by Werkzeug.
department	VARCHAR/TEXT	Default CSE	No	Department or academic unit.
study_year	TINYINT/INTEGER	CHECK 1-6	No	Current study year.
role	ENUM/TEXT	student or admin	No	Access-control role.
created_at	TIMESTAMP/TEXT	Default current time	No	Account creation time.

Clubs

ATTRIBUTE	TYPE	KEY / CONSTRAINT	NULL?	MEANING
club_id	INTEGER / BIGINT	PK, auto-generated	No	Internal club identifier.
name	VARCHAR/TEXT	UNIQUE	No	Club name.
category	VARCHAR/TEXT	-	No	Club type or interest area.
description	TEXT	-	No	Description of club purpose.
created_by	FK -> Users	ON DELETE SET NULL	Yes	Account that created the club.
created_at	TIMESTAMP/TEXT	Default current time	No	Club creation time.

ClubMemberships

ATTRIBUTE	TYPE	KEY / CONSTRAINT	NULL?	MEANING
user_id	INTEGER / BIGINT	PK part, FK -> Users	No	Member account.
club_id	INTEGER / BIGINT	PK part, FK -> Clubs	No	Joined club.
joined_at	TIMESTAMP/TEXT	Default current time	No	Time membership started/reactivated.
status	ENUM/TEXT	active or left	No	Current relationship state.

Events

ATTRIBUTE	TYPE	KEY / CONSTRAINT	NULL?	MEANING
event_id	INTEGER / BIGINT	PK, auto-generated	No	Internal event identifier.
title	VARCHAR/TEXT	CHECK length 3-120	No	Event title.
description	TEXT	-	No	Event details.
event_date	DATETIME/TEXT	Indexed	No	Start date and time.
location	VARCHAR/TEXT	-	No	Venue or location.
capacity	INTEGER	CHECK 1-10000	No	Maximum registrations.

ATTRIBUTE	TYPE	KEY / CONSTRAINT	NULL?	MEANING
club_id	FK -> Clubs	ON DELETE SET NULL	Yes	Optional hosting club.
created_by	FK -> Users	ON DELETE SET NULL	Yes	Account that created event.
created_at	TIMESTAMP/TEXT	Default current time	No	Record creation time.

EventRegistrations

ATTRIBUTE	TYPE	KEY / CONSTRAINT	NULL?	MEANING
event_id	INTEGER / BIGINT	PK part, FK -> Events	No	Registered event.
user_id	INTEGER / BIGINT	PK part, FK -> Users	No	Registered student.
registered_at	TIMESTAMP/TEXT	Default current time	No	Registration or reactivation time.
status	ENUM/TEXT	registered or cancelled	No	Registration state.

Announcements

ATTRIBUTE	TYPE	KEY / CONSTRAINT	NULL?	MEANING
announcement_id	INTEGER / BIGINT	PK, auto-generated	No	Internal notice identifier.
title	VARCHAR/TEXT	CHECK length 3-120	No	Notice headline.
content	TEXT	-	No	Full announcement text.
audience	ENUM/TEXT	all, students, admins	No	Who may see the notice.
posted_by	FK -> Users	ON DELETE SET NULL	Yes	Account that posted it.
posted_at	TIMESTAMP/TEXT	Indexed	No	Publication timestamp.
expires_at	DATETIME/TEXT	-	Yes	Optional visibility cutoff.

C.4 Normalization

First Normal Form (1NF)

Every field stores one value for each row. Multiple club memberships and event registrations are not stored as comma-separated lists; each relationship is a separate row in an associative table. Repeating groups such as several event IDs inside a user record are avoided.

Second Normal Form (2NF)

The main entity tables use single-attribute primary keys, so non-key attributes depend on the whole key. In ClubMemberships, joined_at and status describe the complete pair (user_id, club_id), not only the user or only the club. In EventRegistrations, registered_at and status describe the complete pair (event_id, user_id).

Third Normal Form (3NF)

Club names/descriptions are stored in Clubs rather than repeated in Events or memberships. Event title/date/location are stored in Events rather than repeated per registration. Account email and profile details are stored in Users, and announcements have a separate relation. Therefore, non-key attributes depend on the key, the whole key, and not on other non-key attributes in the proposed design. Derived values such as registration counts are computed by views/queries instead of stored in Events.

C.5 Example functional dependencies

```
user_id -> full_name, email, password_hash, department, study_year, role, created_at
club_id -> name, category, description, created_by, created_at
event_id -> title, description, event_date, location, capacity, club_id, created_by, created_at
(user_id, club_id) -> joined_at, status
(event_id, user_id) -> registered_at, status
announcement_id -> title, content, audience, posted_by, posted_at, expires_at
```

The proposed schema aims at 3NF for the stored operational data. Candidate keys and dependencies should be re-evaluated if the team adds fields such as club-specific member roles, multiple event sessions, or per-user notification preferences.

C.6 Anomalies avoided by the design

IF INFORMATION WERE REPEATED	POSSIBLE ANOMALY	NORMALISED SOLUTION
Club name and description copied into every event row	Renaming a club would require multiple updates and could leave inconsistent values.	Store club details once in Clubs; Events refers to club_id.
Student name/email copied into each registration	Profile changes would not update old registration rows consistently.	Store the account once in Users; registrations refer to user_id.
Event title and location copied into each attendee record	An event edit would require editing every registration row.	Store event details once in Events; EventRegistrations stores relationship attributes only.
Registration total manually stored on Events	Concurrent registration/cancellation could make the count inaccurate.	Derive active registration count from EventRegistrations using a query/view.

PHASE D

Physical Design

Implement tables and query paths with appropriate constraints and indexes.

D.1 Implementation choices

The local prototype uses Flask with SQLite: SQLite is file-based and requires no separate database server for a classroom demo. The package also includes `sql/schema_mysql.sql` for MySQL 8.0+ to demonstrate the same logical design in MySQL Workbench or phpMyAdmin. The two scripts use engine-appropriate types and syntax; data should be created in the engine selected for the demonstration.

D.2 Constraints in implementation

CONSTRAINT TYPE	EXAMPLES IN CAMPUSCONNECT	PURPOSE
Primary keys	users.user_id; events.event_id; composite pair keys	Prevents duplicate row identity.
Foreign keys	event_registrations.event_id -> events.event_id; club_memberships.user_id -> users.user_id	Prevents orphan relationships.
Unique	users.email; clubs.name	Prevents duplicate login email / club name.
CHECK / ENUM	study_year 1-6; capacity 1-10000; enumerated role and status values	Restricts invalid domain values.
NOT NULL	Required names, emails, event date/location and announcement content	Ensures required values are present.
ON DELETE behavior	Cascade membership/registration; set creator references NULL	Keeps relationship lifecycle predictable.

D.3 Indexes and justification

INDEX	COLUMNS	QUERY / REASON
idx_events_date	events(event_date)	Sort/filter upcoming and past events.
idx_events_club	events(club_id)	List events hosted by a selected club; supports club-to-event joins.
idx_registrations_user_status	event_registrations(user_id, status)	Show a user's active registrations and history.
idx_memberships_club_status	club_memberships(club_id, status)	Count active members and list club membership.

INDEX	COLUMNS	QUERY / REASON
idx_announcements_posted_at	announcements(posted_at)	Retrieve the newest notices first.
idx_users_role	users(role)	Filter accounts by role for dashboard statistics.

The primary-key and unique constraints also create indexes in the supported engines. Indexes should be judged by real query plans and dataset size; for a small class demo the benefit may not be measurable, but the selected indexes illustrate common access patterns.

D.4 Views

VIEW	PURPOSE
upcoming_events_view	Provides future events with club name and a derived active registration count.
event_registration_summary_view	Provides capacity, current registration count, and cancelled-registration count per event.

D.5 Representative CRUD statements

The full scripts are stored under `sql/`. The following examples illustrate the expected query style; use valid primary-key values from the database being demonstrated.

```
-- CREATE: add an event hosted by club 1
INSERT INTO events (title, description, event_date, location, capacity, club_id, created_by)
VALUES ('Database Design Workshop', 'A practical schema-design session', '2099-01-15 14:00:00',
        'Computer Centre', 80, 1, 1);

-- READ: show upcoming events and current registration counts
SELECT * FROM upcoming_events_view ORDER BY event_date LIMIT 20;

-- UPDATE: change an event capacity
UPDATE events SET capacity = 100 WHERE event_id = 1;

-- UPDATE: preserve history when a student cancels registration
UPDATE event_registrations SET status = 'cancelled'
WHERE event_id = 1 AND user_id = 2;

-- DELETE: administrator deletes an event; child registrations cascade
DELETE FROM events WHERE event_id = 1;
```

D.6 Transaction and integrity considerations

The web application checks whether a student has an active registration and compares the active registration count with event capacity before writing a registration. For a production-grade system with concurrent requests, this capacity check should be wrapped in a database transaction with locking/atomic capacity enforcement, because two simultaneous requests could otherwise pass the count check. Administrator-only routes must remain protected on the server; hiding a button in the interface is not sufficient.

PHASE E

Prototype, Development, and Testing

A database-backed website that exercises the major relationships.

E.1 Application architecture

```
Browser (HTML templates + CSS + small JavaScript)
                |
                v
Flask routes / session authentication / role checks
                |
                v
SQLite SQL queries + foreign keys + constraints + indexes + views
                |
                v
instance/campusconnect.db

MySQL deliverable: sql/schema_mysql.sql + sql/crud_examples.sql
```

E.2 Interface inventory

PAGE / ROUTE	MAIN INTERACTIONS	DATABASE TABLES
Home `/`	Upcoming-event cards, statistics, notice preview.	events, clubs, users, event_registrations, announcements
Register `/register`	Create student account with validation.	users
Log in `/login`	Authenticate account and create a session.	users
Events `/events`	Search, upcoming/past tabs, register/cancel.	events, clubs, event_registrations, users
Clubs `/clubs`	Browse clubs, join/leave membership.	clubs, club_memberships
Notice board `/announcements`	Read audience-visible, unexpired notices.	announcements, users
Profile `/profile`	Update profile; view registrations and memberships.	users, event_registrations, events, club_memberships, clubs
Admin `/admin`	Create events/clubs/notices; edit/delete events; delete notices.	events, clubs, announcements, event_registrations

E.3 Authentication and authorisation

- Student registration always creates the role `student`; the public form cannot assign administrator privileges.
- Passwords are stored using Werkzeug password hashing helpers and verified during login.
- Protected actions require an authenticated session; administrator routes check the stored role on the server.
- SQL values from user input use parameter placeholders rather than string concatenation.
- Jinja templates escape text by default; user-submitted descriptions are rendered as text rather than trusted HTML.

E.4 Test plan

TEST ID	TEST / STEPS	EXPECTED RESULT	STATUS
T-01	Run database schema check and inspect core tables/views.	Six tables and two views can be created.	PASS - schema test
T-02	Attempt a duplicate event-registration pair.	Composite primary key rejects the duplicate.	PASS - schema test
T-03	Insert membership with unknown user ID.	Foreign-key enforcement rejects orphan membership.	PASS - schema test
T-04	Insert event with capacity 0.	CHECK constraint rejects invalid capacity.	PASS - schema test
T-05	Create membership and registration, then update statuses.	Membership becomes left and registration becomes cancelled.	PASS - schema test
T-06	Add a registration and query registration summary view.	View reports correct active-registration count.	PASS - schema test
T-07	Register a student, log in, then log out in browser.	Account and session workflow behaves as expected.	RUN LOCALLY
T-08	Log in as student and attempt `/admin` route.	Student is redirected and receives an access message.	RUN LOCALLY
T-09	Log in as admin; create/edit/delete an event.	Changes appear in the event list and persist in database.	RUN LOCALLY
T-10	Register for a full event / cancel registration.	Capacity prevents overbooking; cancellation updates status.	RUN LOCALLY
T-11	Join a club and leave it.	Membership status changes and count updates.	RUN LOCALLY
T-12	Set announcement expiry and audience.	Only audience-visible, unexpired notice appears.	RUN LOCALLY

Verification note: six database-schema tests were executed successfully while preparing this package. The Flask web app could not be launched in the preparation environment because its dependency package was unavailable for installation there. Run T-07 through T-12 locally after installing `requirements.txt`; do not present those rows as passed until your team confirms them.

E.5 Known limitations and possible improvements

- The current project is a course prototype with local demonstration credentials and seeded sample data.
- Before public deployment, add CSRF protection, secure secret management, HTTPS, password reset, rate limiting, and a formal account/data retention policy.
- For concurrent event registration in a production system, enforce capacity atomically within a transaction.
- The app uses SQLite by default for portability; run and demonstrate the MySQL script separately if the instructor specifically expects a MySQL-backed web app.
- The data model can be extended with event sessions, attendance, club officers, notifications, or feedback if the team has time and can update the ER diagram accordingly.

PHASE F

Project Presentation

A suggested narrative for the live demonstration and evaluation.

F.1 Presentation goal

The presentation should show not only the website, but also how the database design supports the user requirements. The live demonstration should connect one visible action to the affected SQL rows, then explain the key constraints and normalization choices.

F.2 Suggested presentation sequence

SLIDE	TITLE	CONTENT / TALKING POINT
1	CampusConnect	Project title, course, institute, team name and members.
2	Problem and objective	Fragmented campus information; one portal for events, clubs, and notices.
3	Scope and user roles	Student workflows and administrator responsibilities.
4	ER model	Explain Users, Clubs, Events, Announcements, and the two associative relations.
5	Relational schema + normalization	Primary/foreign keys, composite keys, and why the design targets 3NF.
6	Physical design	Constraints, indexes, views, and representative CRUD statements.
7	Working prototype	Browse/register for an event, join a club, inspect profile; then show admin edit/publish flows.
8	Testing and conclusion	Show test evidence, acknowledge remaining local test rows, list next steps.

F.3 Suggested live demo script

1. Open the landing page and point out event, club, and notice summaries.
2. Create or use a student account. Register for an event, then open the profile page to show the relationship in registration history.
3. Join a club and show the updated membership state/count.
4. Log out and sign in using the demo administrator account. Create a new event, edit an existing event, and publish an announcement.

5. Open the SQL client and query event_registrations / club_memberships to show the stored rows; query a reporting view to show derived counts.
6. Explain one normalization decision and one index/constraint, then close with test results and limitations.

F.4 Questions the team should be ready to answer

- Why do ClubMemberships and EventRegistrations require composite primary keys?
- Which relationships are one-to-many and which are many-to-many?
- How do you prevent duplicate student emails and duplicate registration pairs?
- Why is registration count derived instead of stored in Events?
- What is the purpose of each index and view?
- What extra protection is required before this prototype is deployed publicly?

F.5 Evidence to capture during the demo

EVIDENCE	WHAT IT PROVES
Home or Events page	The web interface is reachable and reads event rows from the database.
Student profile after registering	A student action creates a row in EventRegistrations and history is readable.
Club page after joining	The user-club relationship is stored and active membership counts can change.
Admin edit form and updated event	Administrator access control and the UPDATE path work.
SQL query result or reporting view	Stored relationships, constraints, and derived counts are backed by SQL.

PHASE APPENDIX

Setup and submission checklist

Complete these details and verify each workflow before submitting.

Appendix A. Local setup

1. Install Python 3.10+ and open the project folder in VS Code.
2. Create a virtual environment: `py -m venv .venv`.
3. Activate it and install requirements: `python -m pip install -r requirements.txt`.
4. Start the app: `python app.py`; open `http://127.0.0.1:5000`.
5. Use the seeded admin account for the admin demo and create a separate student account through the signup page.
6. Run the SQL schema tests: `python -m unittest discover -s tests -v`.

Appendix B. SQL artefacts

FILE	USE
sql/schema_sqlite.sql	Local Flask application database; creates tables, indexes, and views.
sql/schema_mysql.sql	MySQL 8+ database DDL with tables, constraints, indexes, and views.
sql/crud_examples.sql	Examples for create/read/update/delete and status updates.
erd.dot	Graphviz source used to generate the ER diagram.

Appendix C. Submission checklist

- [ ] Enter final registered team name and all member names/roll numbers.
- [ ] Confirm the final scope matches the topic registered in the DBMS project sheet.
- [ ] Run the website on the machine that will be used for evaluation.
- [ ] Execute and review the MySQL schema in the tool agreed with the instructor (if applicable).
- [ ] Complete browser tests T-07 through T-12 and record evidence/screenshots.
- [ ] Update this report if features or schema change; keep the ER diagram consistent with the tables.
- [ ] Rehearse the presentation within the allotted time and verify that view access to any shared submission files is enabled.

Appendix D. Assumptions

The registered project topic is CampusConnect. Specific team details and faculty details were not supplied when this package was generated, so explicit placeholders remain. The course brief did not prescribe the precise campus workflows or a technology stack; the feature set and stack documented here are a proposed

implementation of the topic and should be confirmed by the team. The project should be revised rather than submitted unchanged if the registered requirements or faculty directions differ.